

Flake8 & Bandit Security Audit Report

medical-handwriting-ocr & telegram-forwarder

DrAbdulmalek / OmniMedical Suite

2026-06-29

Task 3 of 5: Code Quality & Security Linting

1. Executive Summary

This report presents the results of a comprehensive code quality and security audit performed on two repositories from the OmniMedical Suite ecosystem: **medical-handwriting-ocr** and **telegram-forwarder**. The audit was conducted using two industry-standard Python linting tools: **flake8** for style and code quality, and **bandit** for security vulnerability detection. Both tools were run with a maximum line length of 120 characters and excluded non-Python directories such as frontend, Terraform, Kubernetes, and test directories from the bandit scan.

The medical-handwriting-ocr repository is a large, complex project containing multiple sub-projects (IntelliFile-app, OmniFile-Processor, and a FastAPI backend). It exhibited significant code quality issues with **7,057 flake8 violations** and **1,172 bandit findings**, including 26 HIGH severity security issues. The telegram-forwarder repository is a smaller, focused project for Telegram message forwarding, with **167 flake8 violations** and only **62 bandit findings**, all of LOW severity except one MEDIUM. The findings are detailed in the sections below with specific file locations, issue categories, and recommended remediation priorities.

7,057

Flake8 Issues medical-handwriting-ocr

1,172

Bandit Findings medical-handwriting-ocr

167

Flake8 Issues telegram-forwarder

62

Bandit Findings telegram-forwarder

2. Flake8 Code Quality Analysis

Flake8 is a Python linting tool that enforces PEP 8 style guidelines, detects syntax errors, and identifies code smells such as unused imports, undefined variables, and naming convention violations. The analysis was performed with a maximum line length of 120 characters and excluded build artifacts, virtual environments, frontend code, and infrastructure-as-code directories. The following subsections present detailed findings for each repository.

2.1 medical-handwriting-ocr

The medical-handwriting-ocr repository contains a substantial number of style and quality issues spread across hundreds of files. The dominant issues are whitespace-related (blank lines containing whitespace and missing newlines at end of file), followed by unused imports which suggest significant dead code accumulation over the project's development lifecycle. There are also several critical F821 (undefined name) errors that indicate potential runtime bugs in production code paths.

2.1.1 Top 15 Issue Categories

#	Code	Description	Count
1	F401	Imported but unused	168
2	W293	Blank line contains whitespace	2,735
3	E302	Expected 2 blank lines, found 1	44
4	E226	Missing whitespace around arithmetic operator	39
5	W291	Trailing whitespace	19
6	E402	Module level import not at top of file	13
7	F821	Undefined name	12
8	F811	Redefinition of unused variable	11
9	E127	Continuation line over-indented	9
10	E128	Continuation line under-indented	8
11	F841	Variable assigned but never used	7
12	F541	f-string missing placeholders	7
13	E203	Whitespace before colon	6
14	E722	Bare except clause	3
15	E712	Comparison to True/False	3

2.1.2 Critical: F821 Undefined Name Errors

These are the most concerning flake8 findings because they represent actual runtime errors where code references variables or names that were never defined in the current scope. These will cause `NameError` exceptions when the affected code paths are executed in production. The affected files and locations are listed below:

File	Line	Undefined Name
backend/app/config.py	20	os
backend/app/clinical/guideline_tracker.py	244	s
backend/app/media/speaker_diarization.py	211	Path
backend/app/media/speaker_diarization.py	274	TranscriptionResult
backend/app/media/video_processor.py	411	TranscriptionResult
backend/app/parsers/equation_parser.py	38, 208, 218, 330, 371	Dict

2.2 telegram-forwarder

The telegram-forwarder repository shows a different pattern of issues compared to medical-handwriting-ocr. The most prevalent problems are formatting-related, specifically inconsistent spacing around operators (E221: multiple spaces before operator) and after colons/commas (E241: multiple spaces after colon). These are concentrated primarily in the app.py and forwarder.py files, suggesting a different coding style or manual alignment approach was used.

There are also 6 unused global statements (F824) that should be removed, and several E712 anti-patterns in tests (comparing to True/False instead of using "is" or "if not").

2.2.1 Top 10 Issue Categories

#	Code	Description	Count
1	E221	Multiple spaces before operator	51
2	E241	Multiple spaces after colon/comma	37
3	E251	Unexpected spaces around keyword equals	24
4	E712	Comparison to True/False should use is/if not	11
5	W293	Blank line contains whitespace	23
6	F824	Unused global statement	6
7	E302	Expected 2 blank lines, found 1	2
8	E305	Expected 2 blank lines after definition	1
9	E402	Module level import not at top of file	1
10	F541	f-string missing placeholders	1

3. Bandit Security Analysis

Bandit is a Python security linter designed to find common security vulnerabilities in Python code. It checks for issues such as SQL injection, hardcoded passwords, use of insecure functions, shell injection risks, and more. Each finding is categorized by severity (HIGH, MEDIUM, LOW) and confidence (HIGH, MEDIUM, LOW). The sections below detail the findings for each repository, organized by severity and issue type.

3.1 medical-handwriting-ocr - Security Findings

The medical-handwriting-ocr repository has a significant number of security findings, with 26 rated as HIGH severity. The most critical issues include SQL injection vectors (B608), shell injection through subprocess calls (B605), use of weak cryptographic hashes (B324), and use of the exec() function (B102). Additionally, there are widespread issues with unsafe Hugging Face model downloads without revision pinning (B615) and numerous try/except/pass blocks that silently swallow exceptions (B110). The breakdown by severity and issue type is presented below.

3.1.1 Severity Distribution

Severity	Count	Percentage
HIGH	26	2.2%
MEDIUM	241	20.6%
LOW	905	77.2%
Total	1,172	100%

3.1.2 HIGH Severity Issues (26 findings)

These are the most critical security findings that require immediate attention. They include shell injection vulnerabilities through subprocess calls with shell=True, use of weak MD5 hashing without the usedforsecurity=False flag, and an exec() call that could allow arbitrary code execution. The specific details are:

Test ID	Issue	Count	Example File
B324	Weak MD5 hash (no usedforsecurity=False)	19	OmniFile-Processor/modules/core/file_fingerprint.py:105
B605	Shell injection via shell=True in subprocess	10	OmniFile-Processor/config.py:260-261
B311	Non-cryptographic random for security (random module)	241	Multiple files (IntelliFile-app, OmniFile-Processor)
B102	Use of exec() detected	1	IntelliFile-app/src/core/self_extending_assistant.py:49
B301	Use of pickle (insecure deserialization)	4	OmniFile-Processor modules
B106	Hardcoded password in function argument	4	OmniFile-Processor modules

3.1.3 MEDIUM Severity Issues (241 findings)

MEDIUM severity issues represent significant security concerns that should be addressed as part of a structured remediation plan. The largest category is B615 (unsafe Hugging Face Hub downloads without revision pinning), which could lead to supply chain attacks if model weights are tampered with. SQL injection vectors (B608) in database query construction are also present and particularly

concerning for a medical application handling patient data.

Test ID	Issue	Count
B615	Unsafe HF Hub download (no revision pin)	127
B608	SQL injection via string-based query	49
B104	Binding to all network interfaces (0.0.0.0)	28
B314	Blacklisted XML parse (xml.etree.ElementTree)	12
B405	Import of xml.etree.ElementTree	7
B310	Audit url open for permitted schemes	7
B108	Hardcoded temporary file (mktemp)	7
B107	Hardcoded password in function default arg	7
B403	Import of pickle module	6
B306	Use of insecure mktemp()	4
B614	Insecure YAML load without Loader	2

3.2 telegram-forwarder - Security Findings

The telegram-forwarder repository has a much cleaner security profile compared to medical-handwriting-ocr, with only 62 total findings. The vast majority (61) are LOW severity, consisting of assert statements in the test file (B101, 56 findings) and try/except/pass blocks (B110, 5 findings). The only MEDIUM severity finding is B104 (binding to all interfaces on line 612 of app.py), which is expected behavior for a Gradio application deployed on Hugging Face Spaces. No HIGH severity issues were detected.

Test ID	Issue	Severity	Count	Location
B101	Use of assert (test file)	LOW	56	tests/test_forwarder.py
B110	Try/Except/Pass	LOW	5	app.py:135, forwarder.py:281,428,576,682
B104	Binding to all interfaces	MEDIUM	1	app.py:612

4. Remediation Recommendations

Based on the analysis results, the following prioritized remediation plan is recommended. Issues are organized by urgency level, with critical security vulnerabilities requiring immediate action and code style improvements that can be addressed incrementally through automated tooling.

4.1 Critical Priority (Immediate Action)

These issues represent active security vulnerabilities that could be exploited in production. The B608 SQL injection findings in `medical-handwriting-ocr` are particularly urgent given that this is a medical application that may process sensitive patient data. All string-based SQL query construction should be replaced with parameterized queries using the database driver's parameter binding mechanism.

The B605 shell injection findings in `config.py` and `main.py` should be refactored to use `subprocess.run()` with a list argument instead of `shell=True`. The B102 `exec()` call in `self_extending_assistant.py` should be removed or replaced with a safer alternative such as `ast.literal_eval()` if the use case permits.

4.2 High Priority (Within 1 Week)

The B324 weak MD5 hash findings should be addressed by adding `usedforsecurity=False` to `hashlib.md5()` calls where the hash is used for non-security purposes (file fingerprinting, deduplication), or by migrating to SHA-256 for any security-sensitive hashing. The B615 unsafe Hugging Face downloads should be pinned to specific revisions using the revision parameter in `from_pretrained()` calls to prevent supply chain attacks. The F821 undefined name errors in the backend code should be fixed immediately as they represent actual runtime bugs that will cause `NameError` exceptions when the affected code paths execute.

4.3 Medium Priority (Within 1 Month)

The massive number of unused imports (F401: 168 occurrences in `medical-handwriting-ocr`) should be cleaned up using automated tools such as `autoflake`. The whitespace issues (W293: 2,735 blank lines with whitespace) can be fixed automatically using a code formatter like `autopep8` or `black`. The E302 missing blank lines (44 occurrences) can also be auto-fixed. For `telegram-forwarder`, the formatting issues (E221, E241, E251) can be resolved by running `black` or `autopep8` on the codebase. The `try/except/pass` blocks should be reviewed and either replaced with proper error handling (logging the exception) or removed if the exception is truly expected and safe to ignore.

4.4 Recommended Tooling

To prevent regression and maintain code quality going forward, the following tools and practices are recommended for integration into the development workflow. These tools should be added to the CI/CD pipeline (task 2 from the pending tasks list) to ensure continuous enforcement of code quality and security standards.

Tool	Purpose	Configuration
<code>black</code>	Auto-format code (fixes E2xx, W2xx, E3xx)	<code>line-length = 120</code>
<code>autoflake</code>	Remove unused imports (fixes F401)	<code>--remove-all-unused-imports</code>
<code>isort</code>	Sort imports (fixes E402)	<code>profile = "black"</code>
<code>flake8</code>	Lint check (catch F821, E722, etc.)	<code>max-line-length = 120</code>
<code>bandit</code>	Security scan	<code>--skip B101 (for test files)</code>

Tool	Purpose	Configuration
pre-commit	Run all above before commit	.pre-commit-config.yaml

5. Detailed Bandit Findings - medical-handwriting-ocr

This section provides the complete list of bandit security findings for the medical-handwriting-ocr repository, organized by test ID. Each entry includes the file path, line number, severity, and a brief description of the issue. This information can be used directly by developers to locate and remediate each vulnerability.

5.1 HIGH Severity Detail

B605: Shell Injection via shell=True

File	Line	Description
OmniFile-Processor/config.py	260	Starting process with shell, possible injection
OmniFile-Processor/config.py	261	Starting process with shell, possible injection
OmniFile-Processor/config.py	262	Starting process with shell (seems safe but risky)
OmniFile-Processor/main.py	83	Starting process with shell, possible injection
OmniFile-Processor/main.py	85	Starting process with shell (seems safe but risky)

B324: Weak MD5 Hash

File	Line
OmniFile-Processor/modules/core/file_fingerprint.py	105, 127
OmniFile-Processor/modules/nlp/summarizer.py	248
OmniFile-Processor/modules/nlp/translator.py	243
OmniFile-Processor/modules/vision/ocr_engine.py	1202

B102: Use of exec()

File: IntelliFile-app/src/core/self_extending_assistant.py, Line 49. The exec() function is used to dynamically execute code, which is a significant security risk. If any user input reaches this call without proper sanitization, it could lead to arbitrary code execution. This should be refactored to use safer alternatives such as ast.literal_eval() for parsing, or the design should be restructured to avoid dynamic code execution entirely.

5.2 MEDIUM Severity - SQL Injection (B608)

SQL injection vulnerabilities were found in multiple files where SQL queries are constructed through string concatenation or f-strings rather than parameterized queries. These are particularly concerning in a medical application context where patient data may be stored and queried. Each of these should be refactored to use parameterized queries with placeholders.

File	Line	Context
OmniFile-Processor/database.py	419	String-based query construction
OmniFile-Processor/modules/ai/pattern_db.py	530, 536	String-based query construction
OmniFile-Processor/modules/core/handwriting_db.py	166, 209	String-based query construction
OmniFile-Processor/modules/core/search_engine.py	165, 170, 236, 245, 353	Multiple string-based queries